

Linux de Camões — Instruções de Deploy Automático

Data: 2026-08-01 · **Repositório:** github.com/linuxdecamoes/linuxdecamoes · **VPS:** danieldias@54.37.15.115 (porta 2294)

✓ **O que já está feito (comitado e enviado para o GitHub):** o repositório foi publicado em `master` com a última versão da aplicação (manuais + quizzes, sem pods/terminal) e com toda a infraestrutura de deploy automático. Falta apenas a tua parte na VPS (seção 2) para o CI/CD ficar ativo.

1. O que foi feito

1.1 Versão publicada

- **Manuais LPI:** 6 manuais, 92 tópicos, 146 ficheiros `.mdx` comitados em `frontend/src/content/manuals/`.
- **Quizzes:** 92/92 tópicos com quizzes (460 quizzes) + repetição espaçada SM-2.
- **Sem pods/terminal:** os cartões de "Terminal real em pods Kubernetes" mantêm-se como *em breve* (não há código de pods no repo).
- **Build verificado:** `npm run lint` sem erros + `npm run build` OK (132 páginas estáticas).
- **Zero dados sensíveis no GitHub:** repo é público; todos os `.env` estão no `.gitignore` e só existem na VPS.

1.2 Infraestrutura de deploy (CI/CD)

- `.github/workflows/deploy.yml` — em cada push a `master` : *verify* (lint + build) e depois *deploy* via SSH à VPS (`git reset --hard` + `docker compose up --build -d`).
- **Dependência do Vault externo removida** — o build docker do frontend já não precisa do `../Vault`; builda só com os `.mdx` comitados.
- **Branch canónica** `master` — alinhado o script de setup e o README.
- **Job de deploy resiliente** — se os secrets ainda não existirem, o deploy é ignorado (não falha).
- **Scripts de apoio:** `scripts/setup-vps.sh` (provisionamento), `scripts/backup.sh` (backup local cron), `scripts/backup_s3.py` (S3/R2/MinIO), `scripts/restore.sh` (restauração).
- **Documentação:** guia completo `readme/08-Deploy.md` + README atualizado + histórico no `agents.md`.

1.3 Commits publicados

Commit	Mensagem
2fc12cf	chore(deploy): adicionar scripts de backup e restauro da VPS
ae3600e	docs(vault): registar deploy automatico GitHub para VPS no historico
a8044f7	docs(deploy): criar guia de deploy e backup na VPS

e44c5d0	ci: tornar deploy resiliente a secrets não configurados
fd60f4a	fix(deploy): padronizar branch master no setup-vps e README
2238e00	fix(docker): remover dependencia do Vault no build e run do frontend
0c6c780	fix(ui): resolver erro de lint react-hooks/set-state-in-effect no hero-section

2. O que falta fazer POR TI (na VPS e no GitHub)

⚠ Aviso de segurança: o repositório é público. Nunca coloque chaves de API, tokens ou chaves SSH no GitHub, no README ou em docs. As chaves só vivem na VPS e nos GitHub Secrets (que são encriptados).

Passo 1 — Provisionar a VPS (uma única vez)

Liga-te à VPS e corre **numa só linha** (não quebres o comando em duas linhas — o terminal dá `curl: (2) no URL specified`):

```
curl -fsSL
https://raw.githubusercontent.com/linuxdecamoes/linuxdecamoes/master/scripts/setup-vps.sh |
bash
```

Alternativa mais segura (evita problemas de colagem no terminal):

```
curl -fsSL -o setup-vps.sh
https://raw.githubusercontent.com/linuxdecamoes/linuxdecamoes/master/scripts/setup-vps.sh
bash setup-vps.sh
```

Isto instala Docker + Compose, clona o repo em `/opt/linuxdecamoes` e cria os ficheiros `.env` a partir dos `.env.example`.

Passo 2 — Preencher as chaves reais na VPS

```
vim /opt/linuxdecamoes/.env # GROQ_API_KEY, CLERK_SECRET_KEY,
CLERK_PUBLISHABLE_KEY, POSTGRES_*
vim /opt/linuxdecamoes/backend/.env # DATABASE_URL
vim /opt/linuxdecamoes/frontend/.env.local # NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY,
NEXT_PUBLIC_CLERK_SIGN_IN_URL, NEXT_PUBLIC_API_URL
```

Passo 2b — PostgreSQL: user e password

O user e a password da base de dados são definidos no `docker-compose.yml` (e no `.env` da raiz).

Default: user `kubeai` · password `kubeai` · db `kubeai`. O compose lê de `POSTGRES_USER`, `POSTGRES_PASSWORD` e `POSTGRES_DB` — se não definires no `.env`, usa os defaults.

Importante: a password só é aplicada quando o volume `pgdata` é criado. Se o contentor já arrancou antes, mudar o `.env` **não muda** a password real do PostgreSQL.

- **Sem dados importantes (instalação nova):** apagar o volume e recriar:

```
cd /opt/linuxdecamoes
docker compose down -v
docker compose up --build -d
```

- **Com dados já existentes:** mudar por SQL (não apaga nada):

```
docker exec -it linuxdecamoes-db-1 psql -U kubeai -d kubeai -c "ALTER USER kubeai WITH
PASSWORD 'A_NOVA_PASSWORD';"
```

E tens de alinhar a `DATABASE_URL` **no** `backend/.env` **com a nova password, senão o backend falha a ligação:**

```
DATABASE_URL=postgresql+asyncpg://kubeai:A_NOVA_PASSWORD@db:5432/kubeai
```

Depois de qualquer alteração: `docker compose up --build -d`. Nota: dentro da rede Docker o host da BD é `db` (não `localhost`).

Passo 3 — Subir os serviços e verificar

```
cd /opt/linuxdecamoes
docker compose up --build -d
docker compose ps
```

Esperado: `db`, `backend` e `frontend` **Up (healthy)**.

- Frontend: `http://54.37.15.115:3001`
- Backend API: `http://54.37.15.115:8000`
- PostgreSQL: porta 5432 (local)

Passo 4 — Criar a chave SSH para o GitHub Actions

Na VPS:

```
ssh-keygen -t ed25519 -C "github-actions-deploy" -f ~/.ssh/github_actions
cat ~/.ssh/github_actions.pub >> ~/.ssh/authorized_keys
```

Guarda a **chave privada** (conteúdo completo de `~/.ssh/github_actions`) — vais precisar dela no Passo 5.

Passo 5 — Configurar os GitHub Secrets

No GitHub: **Settings** → **Secrets and variables** → **Actions** → **New repository secret**.

Segredo	Valor
VPS_HOST	54.37.15.115

VPS_USER	danieldias
VPS_SSH_KEY	conteúdo privado de ~/.ssh/github_actions (da VPS)
VPS_PORT	2294
VPS_APP_PATH	/opt/linuxdecamos

Passo 6 — Testar o deploy automático

Faz um push (qualquer alteração) para `master`. Vai a **Actions** no GitHub: o workflow *Deploy para VPS* corre *verify* e depois *deploy*. A VPS é atualizada automaticamente.

```
git add -A; git commit -m "teste: verificar deploy automatico"; git push origin master
```

3. Deploy manual (fallback)

```
ssh -p 2294 danieldias@54.37.15.115
cd /opt/linuxdecamos
git pull
docker compose up --build -d
```

4. Backups (recomendado)

Backup local diário às 03:00 (pg_dump + tar do volume, retenção 7 dias):

```
(crontab -l 2>/dev/null; echo "0 3 * * * /opt/linuxdecamos/scripts/backup.sh >>
/var/log/linuxdecamos-backup.log 2>&1") | crontab -
```

Backup para S3/R2/MinIO e restauro: `scripts/backup_s3.py` e `scripts/restore.sh` (detalhes no guia).

5. Troubleshooting rápido

Problema	Solução
Job de deploy é "skipped"	Secrets ainda não configurados — completar o Passo 5.
Build do frontend falha na VPS	Confirmar que <code>frontend/src/content/manuals/</code> está no repo (o Vault externo não é necessário).
Backend não liga à base de dados (password inválida)	A password só muda com o volume criado — usar <code>ALTER USER ... WITH PASSWORD</code> ou <code>docker compose down -v</code> , e alinhar a <code>DATABASE_URL</code> no <code>backend/.env</code> .
SSH Permission denied	<code>VPS_SSH_KEY</code> errado ou <code>authorized_keys</code> sem a chave pública — refazer Passos 4–5.

Serviço Down no `docker`
`compose ps`

`docker compose logs <serviço>` para ver o erro (geralmente chave em falta no `.env`).

Próximo passo imediato: executar os Passos 1–5 acima. Com os 5 secrets criados e a chave SSH no lugar, o deploy automático fica operacional e cada push atualiza a VPS.

Guia gerado automaticamente em 2026-08-01. Documento completo: `readme/08-Deploy.md` no repositório. · Linux de Camões · Português europeu · Open source.